PROJECT FIELD GUIDE

Interview Tutor

Architecture, design, code, and interview preparation

A practical guide to explaining the project you actually have: how it works, why its components were chosen, what alternatives would change, and where its limits remain.

50 interview questions with harder follow-ups
8 actual source excerpts and 8 coding exercises
Failure scenarios, testing evidence, and improvement priorities

Prepared 5 October 2026

Scope: local single-user Python hobby project. Generated code is not executed. The recorded 68-test baseline uses deterministic mocks. Live account access and grading quality remain unverified.

Contents

1 Project explanation and scope 4

2 Architecture and repository map 6

3 Technology choices and alternatives 7

4 Configuration budgets and cost reasoning 11

5 Code execution model and Python concepts 12

6 Core harness code walkthrough 14

7 Core interview questions 18

8 Sources, security, testing, and PDF 21

9 Security and testing interview questions 25

10 Technology and system design interview questions 28

11 Actual code excerpts and how to explain them 32

12 Coding interview practice 35

13 Debugging and failure scenarios 37

14 Improvements and scaling roadmap 39

15 Mock interview rounds and revision checklist 40

16 Glossary and quick reference 41

17 Source references and maintenance 42

Read sections 1-5 for the overview, 6-11 for implementation and question practice, and 12-17 for coding, debugging, and revision.

How to use this guide

This guide prepares you to explain your local AI interview tutor in a software engineering interview. It connects the product, architecture, Python code, model contracts, research tool, scoring, security boundaries, and tests. It includes suggested answers, challenging follow-ups, and coding exercises. Practise the answers in your own words and open the referenced functions when explaining implementation details.

The implemented project is deliberately small: a user supplies a public page, topic, or pasted material; a Python harness generates a fixed question plan, records answers, requests one final review, and exports a PDF. OpenAI supplies the model responses. Brave supplies bounded snippets only when fresh context is requested and permitted by configuration. Generated Python is educational text and is never executed.

Code snapshot: 4 October 2026. The most recent recorded verification is 68 passing deterministic tests with mocked providers and synthetic credentials. Live provider access and grading quality remain unverified. The explanations of alternatives below are defensible engineering rationales, not a claim that you benchmarked or implemented every alternative.

Use this guide in three passes: first practise the project pitch and architecture; next trace the real code and failure paths; finally rehearse the question banks and write the coding exercises without looking at the hints. An interviewer may ask different questions, so learn the relationships between decisions rather than memorizing a script.

1 Project explanation and scope

A sixty second explanation

"I built a small local Python interview tutor. The user supplies a coding problem, public page, or study topic and selects a practice level. My harness asks the model for a structured interview plan, stores answers locally, and requests an evidence-based review at the end. It computes the final score in Python and renders revision notes as a PDF. For unfamiliar topics, the model can request a bounded Brave search; the harness checks the allowance and uses the returned snippets to replan. I kept the workflow explicit rather than introducing an agent framework. My main engineering work is controlling model inputs and outputs, network access, budgets, failures, and stopping conditions. There are 68 passing mocked tests. The current version does not execute generated code or establish that model grading is accurate."

A three minute explanation

Start with the user problem: reading a solution is different from explaining its prerequisites, applying it to a scenario, and justifying trade-offs. This project gives a learner a structured practice session and revision artifact. It covers coding topics and general concepts such as supervised learning; it does not pretend that every topic requires Python code.

Explain the architecture: one Streamlit application process contains the UI, a `TutorSession` orchestrator, typed response contracts, a model adapter, a constrained source reader, a bounded search adapter, and a local PDF renderer. Session state keeps the plan, transcript, counters, and accepted notes between reruns. There is no database, queue, separate frontend, or worker fleet.

Walk through a normal run: Start validates the material before creating provider clients. The model returns a plan. The harness checks categories, duplicate texts, and the requested internal bounds. Answers are recorded without a model call on every turn. Finishing generates notes and scores once. An accepted score needs a bounded integer and an exact quote from one raw learner answer. The app computes totals and reuses the same accepted notes for the scorecard and PDF.

Walk through the research branch: an insufficient-context response contains a short query and no questions. Python decides whether to search after checking configuration and reserving room for replanning and final review. It accepts at most three snippets, calls the model again with that context, and stops if research is requested again. Search links are citations; the app does not open them automatically.

Finish with trade-offs and evidence: fixed planning keeps cost and control flow simple but sacrifices adaptive probing. In-memory state avoids persistence complexity but loses history on restart. Mocked tests check contracts and failure handling cheaply; they do not replace live quality evaluation. Code is clearly labelled untested because no secure executor exists. The next useful improvements would be calibrated grading and bounded adaptivity, rather than immediately adding unrelated infrastructure.

What is implemented

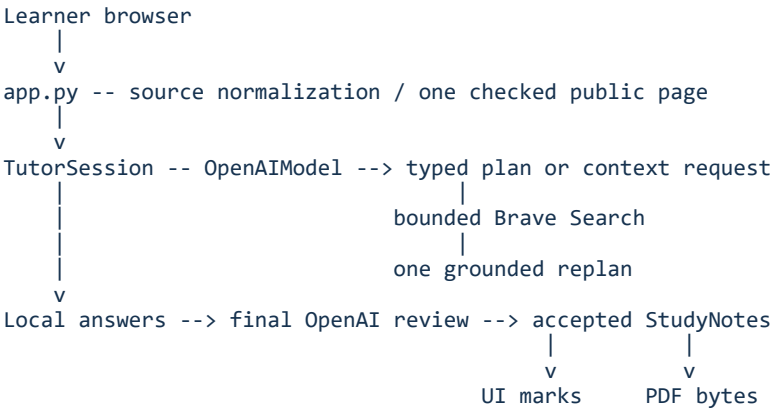
- A direct OpenAI Start flow with URL, topic, paste fallback, level, and company/tier preference.
- One fixed interview plan with four assessment categories and internally enforced bounds.
- Public-page retrieval and optional, configuration-controlled fresh-context search.
- Local answer preservation, final review, evidence checks, usage accounting, and partial exports.
- A PDF containing source material, accepted feedback, relevant reference approaches, and transcript.
- Deterministic unit, integration, headless UI, and mock SDK transport checks.

What you must not claim

- Adaptive per-answer questioning: current questions are fixed after planning.
- Verified brute-force and optimal solutions: reference code and complexity reasoning are untested.
- A secure execution sandbox, repair pipeline, production deployment, or multi-user authentication.
- Objectively accurate hiring-readiness scores or expert-calibrated assessments.
- A precise provider billing cap, guaranteed whole-fetch deadline, or perfect prompt-injection prevention.
- A vector database, trained or fine-tuned model, persistent traces, or durable cross-process idempotency.

The company/tier field is a practice preference. It is not evidence that the app reproduces any employer's actual interview process.

2 Architecture and repository map



The arrows describe data and calls inside one local application. They do not imply separate deployed microservices. OpenAI and Brave are external services. The page reader reaches the one selected public source. The PDF renderer is local and makes no model call.

File or folder	Responsibility	Explain this boundary
`app.py`	Forms, chat UI, Streamlit session, scorecard, download	Events trigger work; ordinary reruns reuse state
`tutor/harness.py`	Requests, state, answers, research, accounting, finish	The program controls actions and stopping
`tutor/schemas.py`	Plan, notes, rubric, safe errors, response wrapper	Structure and local business rules are separate
`tutor/openai_model.py`	Provider SDK, strict schema, safe failure conversion	Provider details stay outside orchestration
`tutor/sources.py`	Public-source retrieval and text extraction	A URL is untrusted input to a network operation
`tutor/search.py`	Fixed Brave endpoint and bounded snippets	Search provenance is not learner evidence
`tutor/secrets.py`	Select one provider credential at runtime	Credentials are authentication, never study context
`tutor/config.py`	Frozen model, price, size, call, and search settings	Configuration is explicit and centralized
`tutor/pdf_export.py`	Render accepted material as PDF bytes	Export does not regenerate the model review
`tests/`	Offline fixtures and covered checks	Test doubles are development tools, not a runtime demo mode
`.streamlit/config.toml`	Loopback binding and UI server settings	Local binding is not authentication
`.gitignore`	Keep keys, environments, caches, and build files out	Ignoring a path does not erase tracked history

Architecture defines component boundaries; design specifies contracts and transitions. The model boundary becomes a typed `ModelReply` that `_request()` checks before acceptance.

3 Technology choices and alternatives

For each choice, explain the requirement it serves, where it appears, the cost it introduces, and the situation that would justify changing it. A technology is not universally best merely because it fits this project. The alternatives here were left outside the current implementation; their discussion is engineering reasoning rather than a historical benchmark report.

Python and a small package

Where: all application modules, `requirements.txt`, and the six test modules. Python gives the project one language for UI orchestration, schema validation, API integration, testing, and PDF generation. A `tutor/` package keeps reusable logic separate from the UI script. It is easier to inspect an ordinary function call than a distributed workflow at this scale.

Alternative: TypeScript with a browser frontend and Node backend would fit a web-focused team or richer client-side UX. Java or Go could suit other backend ecosystems. None automatically improves grading or model reliability. Moving languages now would add setup and translation work without solving the current local-user requirement. Reconsider when a deployment target, team expertise, or measured workload calls for it.

Streamlit instead of a separate frontend and backend

Where: `app.py` uses forms, chat messages, input widgets, session state, and a download button. This is appropriate for a local Python hobby app because it avoids an API server, frontend build, and separate client data model.

Alternative: React plus FastAPI would offer finer UI control, independently usable HTTP endpoints, and a cleaner path to multiple clients. It also introduces request/response APIs, frontend dependencies, cross-service deployment, and authentication decisions. Flask would be lighter but still require templates or a frontend. Consider these when product UX, mobile clients, or multi-user operation becomes a real requirement. They were not needed to demonstrate this harness.

Streamlit reruns the script on interactions, and forms group input submission. This project preserves `TutorSession` and PDF bytes in session state, while work is guarded by submitted events and terminal states. This follows the documented Streamlit execution model and Session State API.

Explicit orchestration instead of an agent framework

Where: `TutorSession.start`, `send`, `finish`, `_request`, and `_research`. The sequence is short and bounded. Ordinary Python makes budgets, permitted tools, and stop conditions visible. A model response proposing research is not permission to execute arbitrary actions.

Alternative: LangChain or LangGraph could organize a larger tool graph, persistence, branching, or human checkpoints. Multiple agents could split specialist roles. Those features bring abstractions, configuration, and additional failure interactions. They are not required for one plan, at most one search, and one review. Reconsider after the workflow gains enough genuinely different branches that explicit code becomes hard to reason about. The current app has no autonomous agent fleet.

One fixed plan instead of per-answer model calls

Where: `InterviewPlan.questions`, `TutorSession.current_question`, and `send`. The fixed plan makes categories and limits checkable before the interview. Normal answer collection has no provider latency or additional generation cost.

Alternative: Adaptive follow-ups would react to mistakes and uncertainty and resemble an interviewer more closely. They require per-turn question contracts, category coverage tracking, evidence-linked follow-ups, deduplication, response budgets, and updated stopping rules. That improvement is meaningful, but it changes the application design. It is future work; the current final notes' three practice prompts are not adaptive live turns.

The configured OpenAI model

Where: `Config.model` is `gpt-4.1-mini`; `OpenAIModel` calls the Responses API. The official model page currently lists Responses and Structured Outputs support, with standard text pricing of $0.40 per million input tokens and $1.60 per million output tokens. These match the configured prices as checked on 4 October 2026. Account access was not tested. See the official model reference.

Alternative: A different small hosted model might cost less or perform better on a particular benchmark. A larger model might improve some difficult reasoning. A local model could change privacy and recurring-cost trade-offs but adds model serving and hardware needs. There is no comparative quality benchmark here, so do not call this the cheapest or best model. The defendable position is a configurable, low-cost starting point supporting the required contract. Compare alternatives with a small labelled evaluation set before changing it; update price assumptions together and never upgrade silently.

Pydantic contracts instead of loose dictionaries

Where: `InterviewPlan`, `CriterionScore`, `InterviewAssessment`, and `StudyNotes`. Local validators enforce the four categories, distinct normalized questions, research/plan mutual exclusion, score maxima, and nullable unassessed marks. The API schema is generated from these models and tightened before submission.

Alternative: Plain dictionaries plus handwritten checks reduce dependencies but scatter contracts and risk missing rules. JSON mode can return syntactically valid JSON without matching the required fields. Marshmallow or JSON Schema validators would be possible substitutes. Pydantic already fits the typed Python flow. Type hints alone do not perform equivalent runtime validation. See Pydantic models and OpenAI Structured Outputs.

There is an important code-level qualification: the plan, question, criterion, and assessment models forbid extra fields locally, while some notes/approach models use Pydantic defaults. The submitted strict API schema forbids extras for its objects. Do not claim every local model has identical strictness or that schema adherence establishes truth.

Dataclasses for configuration and small transport objects

Where: `Config`, `Source`, and `SearchContext` are frozen dataclasses; `ModelReply` is a dataclass carrying typed content and optional usage. These are small internal containers whose fields are already created by application code. Dataclasses avoid boilerplate constructors and give readable defaults.

Alternative: Pydantic Settings would be useful for a larger environment-driven configuration surface with runtime validation. A plain dictionary would be less explicit; a custom class would duplicate boilerplate. `frozen=True` prevents field reassignment, not deep mutation of nested lists. For example, a frozen `SearchContext` can still contain a mutable `sources` list. `Config` also does not validate every conceivable invalid manual price or limit; stronger configuration validation would be a useful future improvement.

The official SDK with explicit metadata handling

Where: `OpenAIModel.__init__`, `generate`, and `_response_format`. The SDK provides typed responses and recognizable exception categories. The adapter still controls the endpoint, timeout, retries, format, and status inspection.

Alternative: Raw HTTP gives transport control but makes the application own request formats, response parsing, authentication, and error conventions. SDK `parse()` is convenient, but this adapter uses `create()` and parses locally after retaining status and usage. This preserves accounting for incomplete or malformed content. Neither option is universally wrong; choose the sequence that preserves the metadata your failure policy needs.

Conditional Brave context instead of searching everything

Where: the plan can contain `research_query`; `_research()` owns allowance and `BraveSearch.search()` owns the fixed endpoint. Stable topics skip research. This avoids unnecessary calls and makes fresh context a bounded extension of planning.

Alternative: Always searching adds cost and may insert irrelevant material into familiar topics. An autonomous browser can inspect pages but expands the action surface. A curated knowledge base adds editorial responsibility. Another search provider could replace Brave through an adapter, but would require new credentials, response parsing, and pricing assumptions. The current one-search snippet workflow is enough for the hobby requirement and is not an unlimited browsing agent.

Source excerpts instead of a vector database

Where: `Source.text`, `research_context`, and `_messages` insert small bounded excerpts directly into the request. There is no document corpus, embedding pipeline, chunk index, or similarity search.

Alternative: A retrieval-augmented system with embeddings is useful for repeated searches over many documents. It adds ingestion, chunking, freshness policies, retrieval evaluation, and storage. Fine-tuning changes model behaviour from examples; it is not a substitute for retrieving current facts. This project adds context at inference time without training a model. Call it bounded retrieval-assisted prompting, rather than claiming a full vector RAG platform.

In-memory state instead of a database

Where: `TutorSession` fields and `st.session_state`. This matches a local, single-user session and avoids schema migrations, connection management, and retention rules.

Alternative: SQLite would be a reasonable next step for saved local runs. PostgreSQL fits a shared service needing stronger multi-user operations. Both require deciding what to store, how to redact it, how to recover partially completed requests, and how to delete user data. Session state is not durable history and does not recover a provider transaction after a crash. Add persistence only when saved history or reliable resumption is requested.

ReportLab instead of browser-based PDF generation

Where: `build_pdf` creates styled paragraphs, code blocks, tables, and page numbers in memory. Python-native rendering integrates without a separate rendering service.

Alternative: HTML-to-PDF can use richer CSS and HTML templates but needs a compatible rendering engine and safe content handling. LaTeX produces strong typesetting but adds toolchain and escaping work for generated content. Beamer is used for the separate architecture presentation, not for runtime revision PDFs. ReportLab fits this simple export pipeline; visual checks are still necessary. See ReportLab Platypus.

Standard-library network and HTML helpers

Where: `http.client`, `socket`, `ssl`, `ipaddress`, `urllib.parse`, and `HTMLParser` in `sources.py`; `http.client` and JSON in `search.py`. The custom page reader can pin the checked public address and revalidate redirects rather than rely on a client's default behaviour.

Alternative: `requests`, HTTPX, Beautiful Soup, or a readability library would improve ergonomics or extraction coverage. They would still need a public-only validated transport; simply validating a URL and then calling a convenience client does not prevent DNS rebinding. The custom code is inspectable, but owns more maintenance responsibility. Small dependency count is not proof of security.

Environment variables and an ignored local key file

Where: `load_key` recognizes provider-specific names, prioritizes environment variables on the default path, and supports the existing named local file format. Provider clients receive only their selected credential. The actual key values are never part of study data or report content.

Alternative: `python-dotenv` would conveniently load `.env` values, but the app already supports runtime environment variables and the user's existing file. A managed secret store is appropriate for deployment, rotation, and access policy, and requires infrastructure absent here. The local file is plaintext; this is not a vault. `.gitignore` prevents new accidental additions, not exposure from already tracked history.

Deterministic unittest checks instead of routine paid calls

Where: `unittest`, `Mock`, `patch`, temporary synthetic files, `AppTest`, and an SDK mock transport. Known fixtures make failure paths repeatable and keep routine checks independent of credentials, provider availability, and billing.

Alternative: pytest can provide concise fixtures and parameterization; it would not inherently improve grading accuracy. Live smoke tests check actual provider access but add cost and nondeterminism. Human-labelled evaluation checks quality but requires independent judgements. The current tests and future live evaluations serve different purposes. There is no CI pipeline claimed here.

Pinned requirements and development-only PDF reading

Where: runtime requirements pin Streamlit, OpenAI, Pydantic, and ReportLab; `requirements-dev.txt` adds pypdf. Pinning makes the observed environment more reproducible than unconstrained version ranges. pypdf checks exported text in tests; it does not generate the runtime report.

Alternative: A lockfile with a dedicated environment manager would pin transitive dependencies more completely. Exact top-level versions alone do not guarantee bit-identical environments. Separating development requirements avoids making test-only PDF inspection an application feature. `.venv` isolates packages, not malicious code, and is not a sandbox.

4 Configuration budgets and cost reasoning

Control	Current value	Purpose
Model	`gpt-4.1-mini`	Configurable structured generation
Estimated session allowance	$0.05	Includes model requests and search estimate
Model-call ceiling	12 attempts	Failed attempts count; not interview question progress
Input allowance	40,000 UTF-8 bytes	Serialized context/schema plus local overhead
Source excerpt	4,000 characters	Bound retained page text
Source/problem allowance	8,000 characters	Harness material check
Answer allowance	1,000 characters	Reject overlong answers before acceptance
Planning output	1,800 tokens	Bound plan/context-request output
Report output	3,500 tokens	Bound grading and educational notes
Search attempts	1 by default	Set `max_search_calls=0` to disable
Search context	4,000 characters	Bound grounding snippets
Search result count	At most 3	Small provenance set
Page/search response body	1,000,000 bytes	Count streamed bytes, not headers alone

`Config.cost` uses `(input_tokens * input_price + output_tokens * output_price) / 1_000_000`. Token counts are not character counts. The local preflight input byte estimate is deliberately conservative but remains an approximation including a fixed overhead allowance.

An illustrative normal run assumes 2,000 input and 1,000 output tokens for planning, then 6,000 input and 3,000 output for review. Planning costs $0.0024 and review costs $0.0072, for $0.0096 total under configured prices. PDF rendering is local; the review already contains PDF explanation content. This is arithmetic, not a measured invoice.

An illustrative research run assumes 2,000 input/150 output for the context request ($0.00104), one search ($0.005), 4,000 input/1,000 output for replanning ($0.0032), and 8,000 input/3,000 output for review ($0.008). Total: $0.01724. No retries or cached-input discounts are assumed. Brave's public Search plan lists $5 per 1,000 requests; the local estimate ignores credits and account eligibility. See Brave pricing.

The practical interview answer is: "I bound calls and response sizes, reserve allowance before dispatch, replace reservations with reported usage when available, and expose uncertainty. I do not present the estimated budget as an exact provider billing guarantee." Prices can change, request accounting can be incomplete, and external account credits and project limits are separate.

5 Code execution model and Python concepts

What runs and what is only text

The application's own trusted Python runs Streamlit, validation, requests, and rendering. The model's generated Python is stored as an `Approach.code` string and displayed. There is no `exec`, evaluator, subprocess executor, sandbox, or repair loop for those solutions. A code example in this guide explains the harness; a proposed exercise is clearly marked as future practice. Neither is evidence of a tested learner solution.

State and indexing

`len(self.answers)` selects the current question. One accepted answer advances one position. Validation occurs before appending. This avoids keeping a separate question index and answer count synchronized. The questions and raw answers remain different collections because scoring must refer to learner evidence, while messages are a display transcript.

Types and validation

`Literal` restricts recognized category labels in Pydantic. `str | None` communicates nullable data. `Field` defines size/range rules; an after-validator can inspect relationships between fields. `strict=True` on score integers rejects coercion such as numeric strings and booleans. A Python type annotation on an ordinary function or dataclass is not by itself a runtime enforcement mechanism.

`@property` exposes computed values such as `total` and `current_question` without another stored field. `None` represents unassessed data. Computation avoids a model-supplied aggregate drifting from individual criterion marks. Frozen dataclasses reduce accidental field reassignment; nested mutable containers still require care.

Serialization and messages

`json.dumps` serializes source and answers as data in the user message; system instructions separately describe the intended task. `ensure_ascii=False` preserves natural Unicode, while encoding to UTF-8 measures bytes for the local bound. JSON encoding is not a security proof against model prompt injection; content remains capable of influencing a language model.

Exceptions and cleanup

Custom exceptions carry safe categories and messages. `raise ... from None` suppresses an exception chain in the user-facing path; it is not a general policy preventing every debugger or runtime from seeing internal data. `finally` closes network connections even when parsing or limits fail. `time.monotonic()` measures elapsed deadlines without relying on the wall clock being stable. `datetime.now(timezone.utc)` supplies a date in planning context; this does not create current facts by itself.

Local computation complexity

Let `B` be downloaded bytes, `T` retained text length, `Q` planned questions, and `A` total answer characters. Extraction is roughly linear in the downloaded input, with bounded retained context. Normalized duplicate checking builds a set over question strings. Exact evidence checking searches each assessed quote within raw answers; the constants are small and the input is bounded, but substring-search details depend on the implementation and pattern lengths.

Serializing report context requires reading the accumulated material and answers, so it is not free even when no model is called. Provider generation latency is not explained by calling Python orchestration O(1). It depends on request size, output size, service conditions, and model behaviour; no latency benchmark was

measured. Algorithm complexity claimed for generated reference solutions is a separate educational claim and is not validated by these local control-flow tests.

Why two kinds of consistency are different

Using the same accepted `StudyNotes` object makes app/PDF scores consistent. It does not mathematically ensure the explanation matches the generated algorithm or that the algorithm is correct. Prompts ask for consistent prose, dry runs, and complexity, but a stronger evaluator would need independently specified checks and possibly secure execution. Be precise about which consistency is checked by code.

6 Core harness code walkthrough

This section explains the implemented application. Reasons below are engineering justifications for the current design, rather than a claim that every alternative was formally evaluated before coding. The project is a small local interview-practice application: it plans questions, records answers, asks the model for a final review, and renders revision notes. It does not run generated Python, repair failed implementations, predict hiring success, or conduct an adaptive conversation after every answer.

The clean Start flow

`app.py` owns the Streamlit page. The initial form accepts a full public-page URL, a topic, optional pasted material, practice level, and optional company/tier. **Start interview** immediately begins the workflow. There is no demo selector, provider selector, pricing block, search checkbox, or spending-approval checkbox. OpenAI is the runtime model provider; research is available automatically when configuration permits it and the planning response requests it.

The order matters. `load_source()` validates and obtains material before `OpenAIModel(config.model)` and `BraveSearch()` are constructed. A blank or invalid submission therefore does not construct those clients. The UI stores the new `TutorSession` in `st.session_state`, invokes `start()`, and reruns the page. Ordinary reruns find the existing object; they do not start a second interview. A compatibility check offers a reset if an older in-memory object lacks required fields or uses a different configured model.

After submission, the page shows source material, recorded messages, and any notices. It shows a chat input only while the status is `Interviewing`. Terminal sessions show the scorecard and download button. The app computes PDF bytes once and retains them in session state. **New session** clears both the session object and cached PDF.

Proof: Start form, session-state, and PDF-cache branches in `app.py`; `tests/test_tutor.py::test_app_has_clean_start_and_defers_clients_until_valid_submission`.

What the harness controls

The model proposes content; ordinary Python decides which actions are permitted. `TutorSession` is the harness because it owns state, validates inputs, constructs requests, counts attempts, reserves budgets, calls the model/search interfaces, accepts or rejects responses, and determines when work stops.

The central methods are:

Method	Responsibility	Important boundary
`start()`	Obtain a plan, optionally research and replan, show the first question	Only allowed from `Ready`
`send(text)`	Validate and preserve one answer; show the next question or finish	Only allowed during `Interviewing`
`finish()`	Generate final notes and validate assessment evidence	Terminal repeats do nothing
`_request(report=...)`	Check input/call/cost limits, call the model, account usage	Budgets checked before dispatch
`_research(query)`	Check allowance, call search once, accept bounded snippets	Failed searches still count
`_messages(report)`	Build trusted instructions plus JSON data	Source/answers are reference data

`TutorSession` receives its model, configuration, and optional search client through its constructor. This is dependency injection: the same orchestration runs with an OpenAI adapter at runtime and small deterministic stand-ins during tests. A framework is unnecessary for this short, explicit workflow.

Actual states and transitions

The state starts as `Ready`. A successful `start()` assigns the accepted plan, moves to `Interviewing`, and displays a question. Planning, searching, and replanning happen synchronously inside `start()`; there is no separate persisted `Planning` state.

`send()` appends a validated answer. If another question remains, it displays it. If none remains, it calls `finish()`. The user can also finish early. With answers available, `finish()` temporarily sets `Finishing`, requests notes, validates evidence, and ends as `Finished - unverified` after full coverage or `Partial - unverified` after early completion. Without answers it stops as partial, provides a not-assessed notice, and makes no grading request.

Planning, research, report, or context-limit failures also lead to a partial export. The word unverified is intentional: generated code is not executed and AI explanations are not independently proven. A completed workflow is different from a verified solution.

Walkthrough 1: planning and collecting answers

The following is a shortened teaching version of the actual control flow, not a replacement implementation:

```
def start(self):
    require(self.status == "Ready")
    plan = self._request()
    if plan.research_query is not None:
        self._research(plan.research_query)
        plan = self._request()
        require(plan.research_query is None)
    require(len(plan.questions) == self.question_count)
    self.plan = plan
    self.status = "Interviewing"
    self._show_question()
```

The real method handles `ValueError`, records a safe notice, and changes status to partial. A second request for research is rejected rather than recursively searching. The model can request context, but it cannot directly invoke a browser or choose unlimited tools.

`current_question` uses `len(self.answers)` as the index into the accepted plan. There is no separate mutable index to keep synchronized. `_show_question()` appends an assistant message containing the current question number and category. It does not announce the planned total. The plan is generated upfront: recording an intermediate answer makes **zero** model calls. This reduces latency, expense, and failure opportunities, while sacrificing answer-dependent probing.

`send()` strips outer whitespace and rejects blank answers or answers over `Config.max_message_chars`. It does not truncate a long answer into a misleading accepted answer. After acceptance, it appends to both raw `answers` and displayed `messages`. It then checks the future report input bound. If that bound is too large, the entire latest accepted answer remains available in a partial export and grading stops.

Proof: `TutorSession.start`, `send`, and `current_question` in `tutor/harness.py`; tests `test_answers_use_no_model_calls_and_finish_runs_once`, `test_context_failure_preserves_full_latest_answer_and_stops`, `test_answer_limits_reject_invalid_input_without_recording_it`.

Walkthrough 2: request guards and accounting

`_input_bound()` counts UTF-8 bytes for serialized messages and the local JSON schema, then adds 1,024 bytes of overhead. This is a conservative local planning measure, not a model tokenizer or provider-certified maximum bill. Non-ASCII characters can consume more bytes than their character count suggests.

Before dispatch, `_request()` checks this bound, selects the configured output-token ceiling, estimates an attempt reservation, and preserves room for final notes when the request is a plan. It also preserves a final model-call slot. Calls and estimated spending are incremented **before** `generate()` so failed attempts cannot evade accounting.

```
reservation = config.cost(input_bound, output_limit)
calls += 1
estimated_cost += reservation
try:
    reply = model.generate(messages, report=report,
                           max_output_tokens=output_limit)
except ModelRequestError as error:
    _record_usage(error, reservation)
    stop_with_safe_partial_result()
_record_usage(reply, reservation)
accept_only_expected_contract(reply.content)
```

When both token counts are known, `_record_usage()` replaces the reservation with the configured-price estimate for reported tokens. When usage is missing, it retains the reservation and marks `usage_unknown`. This avoids pretending that an uncertain failed request cost zero. Current configuration uses `gpt-4.1-mini`, a 12-model-call ceiling, a $0.05 estimated session ceiling, 40,000 input bytes, and output ceilings of 1,800 tokens for planning and 3,500 for notes. These are configuration values, not a fresh official pricing verification or an exact account-level financial cap. The application never upgrades models automatically.

Walkthrough 3: structured output and failures

`OpenAIModel.generate()` chooses `InterviewPlan` or `StudyNotes`, obtains JSON Schema from Pydantic, and calls `responses.create()`. `_response_format()` recursively removes defaults, makes every object property required, and sets `additionalProperties=False`. Nullable fields remain nullable but must appear, such as `research_query: null`. These API object rules are distinct from local semantic rules.

The adapter reads usage before attempting local JSON validation. It rejects incomplete responses, detects output-limit termination, checks refusal content, and rejects empty output. Only then does `schema.model_validate_json()` produce a typed object. An exact credential echo is replaced before the object can reach the UI or report. No raw provider error body or generated malformed text is copied into a user-facing error.

Using `responses.create()` instead of the SDK's convenient `responses.parse()` is purposeful here: premature parsing can raise on truncated or invalid JSON before the adapter receives usage metadata. The explicit order makes failure accounting testable. `_failure_kind()` classifies authentication, quota, rate limits, permission, model access, schema/request errors, provider availability, connection, and timeout. `ModelRequestError` contains a fixed safe message plus optional token counts, not the original provider exception.

`InterviewPlan.validate_questions()` enforces distinct normalized question texts and all rubric categories. A research request must contain an empty question list; a normal plan must meet the bounded plan contract. The harness separately checks the exact requested internal length. Structured JSON alone cannot guarantee relevant, factual, well-calibrated questions.

Proof: `OpenAIModel.generate`, `_response_format`, and `_failure_kind` in `tutor/openai_model.py`; `InterviewPlan` in `tutor/schemas.py`; tests `test_incomplete_response_is_checked_before_json_parsing`,

`test_sdk_mock_http_response_retains_truncated_output_usage`, `test_semantic_plan_validation_remains_enforced`.

Walkthrough 4: scoring and finalization

The rubric is preparation demonstrated (3), concept knowledge (3), scenario application (2), and reasoning/clarity (2). These sum to 10. Strict integer fields reject numeric strings and booleans. The application computes `total`; the model cannot submit a fabricated aggregate. A missing criterion is `marks=None`, so the overall total is also `None`. Missing evidence is different from an observed incorrect answer earning zero.

`finish()` validates each assessed criterion's evidence against raw learner answers. The stripped quote must be a nonempty exact substring of **one** answer. Text from the webpage, a search snippet, a tutor question, or two concatenated answers cannot independently satisfy that rule. If any evidence fails, the whole assessment is removed; educational notes remain available with a not-assessed notice.

This protects provenance and range bounds, not grading truth. A real quote can still receive an unfair score or poor rationale. The current project has no expert-calibrated grading dataset or semantic judge. `Finished - unverified` can therefore coexist with no accepted assessment.

Finalization is idempotent at the session level: calls to `finish()` from terminal statuses return immediately. Streamlit reruns cannot purchase a fresh report merely by redisplaying the page. This is not a durable cross-process idempotency mechanism; server restart loses the in-memory session. The UI renders the same accepted `StudyNotes` object used for the PDF and caches PDF bytes separately.

Illustrative traces to explain at a whiteboard

Stable topic, no research: the user enters supervised learning and submits. Source loading produces material; clients and a session are created. Planning returns a valid `InterviewPlan` with no research query. The harness displays questions and stores answers locally. The user finishes or reaches completion. One final report request returns `StudyNotes`; evidence is checked; the UI and PDF use those notes. The ordinary path has two model attempts and no search. This is a control-flow illustration, not a claim that a live run was performed.

Fresh context required: planning returns a short topic query and no questions. `_research()` checks the search allowance and reserves room for a replan and report, counts the search attempt, and obtains bounded snippets/provenance. The next planning payload includes that context and reports research unavailable. One accepted replan continues the interview. A second research request stops. The usual successful research path has three model attempts and one search; linked result pages are not automatically opened.

Final report fails: learner answers already exist. `finish()` attempts one report request. If it times out, returns truncated JSON, or fails validation, the harness keeps the answers, records safe failure information and available usage, and ends partial. The PDF can still export available content without another model request. It must not invent marks or present a failed report as validated. Missing-usage accounting retains the attempt reservation.

7 Core interview questions

Q01. What makes this an AI harness rather than a chat wrapper?

Answer: Python controls state, inputs, output contracts, optional research, accounting, and stopping. The model supplies bounded proposals and review content.

Follow-up: Does the model control the workflow? It requests research through one field; `start()` and `_research()` decide whether it can happen. It cannot call arbitrary tools. **Proof:** `tutor/harness.py::start`, `_research`, `_request`.

Q02. Why use one `TutorSession` object?

Answer: It keeps the plan, answers, status, counters, notes, and provenance together, making transitions explicit and testable.

Follow-up: Is that good for many users? Streamlit session state separates browser sessions, but this is not durable storage or a complete multi-user backend. **Proof:** `app.py` session-state handling; `TutorSession.__init__`.

Q03. Why pass the model into the constructor?

Answer: Dependency injection separates orchestration from vendor calls and allows deterministic failure/usage tests.

Follow-up: Could another provider be swapped in instantly? A new adapter must preserve `generate(...)`, typed content, safe errors, and usage semantics; its schema capabilities must also be checked. **Proof:** `TutorSession._request`; `ModelReply`; test-only `NotesModel`.

Q04. Why generate the interview upfront?

Answer: One plan request makes normal answer collection cheap, fast, and predictable. Categories and stopping bounds are known before the conversation proceeds.

Follow-up: Are follow-ups adaptive? Current interview questions are fixed after planning. The final notes include three revision follow-ups. Per-answer probing is a future change requiring new state, evidence, and budget rules. **Proof:** `current_question`, `send`, `REPORT_INSTRUCTIONS`.

Q05. Why use JSON Schema and Pydantic together?

Answer: The API schema constrains response shape; Pydantic parses types and enforces local business rules.

Follow-up: Does strict JSON prevent hallucinations? No. Categories, ranges, and duplicates are locally checkable; factual correctness and grading quality need separate evaluation. **Proof:** `_response_format`; `InterviewPlan.validate_questions`.

Q06. Why call `responses.create()` instead of `parse()`?

Answer: The adapter preserves response status and usage before local parsing, including malformed or truncated outputs.

Follow-up: Is `parse()` bad generally? No; it is convenient when its failure behavior matches application needs. Here metadata-aware accounting is more explicit. **Proof:** `OpenAIModel.generate`; `test_sdk_mock_http_response_retains_truncated_output_usage`.

Q07. Why make nullable properties required in the schema?

Answer: Strict API object rules require all properties; `null` explicitly communicates absence without omitting a key.

Follow-up: What happens to local defaults? Defaults are removed from the submitted schema, while the Pydantic models still provide local semantics. **Proof:** `_response_format`; `test_strict_schema_format_keeps_nullable_fields_and_object_rules`.

Q08. Why disable automatic SDK retries?

Answer: One harness-counted attempt should not silently generate multiple transport attempts with uncertain cost or delay.

Follow-up: Would retries improve availability? Often yes. An explicit bounded retry policy could distinguish transient errors from authentication/quota errors and account every attempt; it is not implemented now. **Proof:** `OpenAIModel.__init__`, `max_retries=0`; failure tests assert one call.

Q09. How do you stop repeated report generation?

Answer: `finish()` returns outside `Ready`/`Interviewing`; terminal status plus cached PDF prevents ordinary rerun repetition.

Follow-up: What about a crash after a provider accepted the request? In-memory guards cannot recover that transaction. Durable request/run records would be needed. **Proof:** `TutorSession.finish`; `app.py` PDF caching.

Q10. Why reserve money before making a request?

Answer: Unknown usage and failed attempts should still consume allowance. Planning also leaves room for the final report.

Follow-up: Is $0.05 an exact financial cap? No: prices are configured estimates, bytes approximate input tokens, and provider/account behavior is external. Call/output bounds are clearer hard controls. **Proof:** `_request`, `_record_usage`, `Config.cost`.

Q11. Why count failed searches?

Answer: An attempted external search may incur cost or consume allowance even when the application does not receive usable results.

Follow-up: Can it keep searching until it finds something? No. One configured attempt is allowed by default and a repeated research request stops. **Proof:** `_research`; `test_failed_search_is_counted_sanitized_and_never_retried`.

Q12. Why reject long answers instead of truncating them?

Answer: Truncation could remove important reasoning while grading the user as though the full answer was reviewed.

Follow-up: What if total context grows too large after accepting an answer? The raw latest answer remains intact, grading stops, and a partial export is available. **Proof:** `send`; `test_context_failure_preserves_full_latest_answer_and_stops`.

Q13. How do you prevent fabricated marks?

Answer: Bound strict integer scores, compute the aggregate locally, and require an exact quote from one recorded answer for every assessed criterion.

Follow-up: Does a quote prove the mark is right? No. It proves the cited text came from the learner; semantic calibration remains unverified. **Proof:** `InterviewAssessment.validate_marks`, `validate_evidence`, `total`.

Q14. Why use `None` for unassessed criteria?

Answer: Missing evidence should not be confused with an incorrect demonstrated answer worth zero. An incomplete rubric has no total.

Follow-up: Why not score only the available categories? That would change the scale and risk misleading comparisons; the present rubric keeps the same denominator and labels partial feedback. **Proof:** `InterviewAssessment.total`; `app.py` scorecard branch.

Q15. How do you handle prompt injection?

Answer: Trusted instructions are separate from JSON source/answer data, prompts label that data untrusted, and Python validates the permitted contracts/actions.

Follow-up: Is injection solved? No. Content can still influence the model's prose or judgment. Local bounds reduce consequences, while exact evidence prevents source-only scoring. **Proof:** `_messages`, instruction constants, `validate_evidence`.

Q16. Why are error messages fixed strings?

Answer: Provider errors can contain credentials, raw request fragments, or unsafe details. Typed categories give actionable safe messages without exposing those bodies.

Follow-up: How do you debug without raw details? Current categories and usage narrow the cause; richer sanitized telemetry is future work. Do not claim persistent traces exist. **Proof:** `_failure_kind`; `ModelRequestError`; safe-error tests.

Q17. Why not execute reference code?

Answer: The current hobby scope reviews and documents code without a secure executor. An ordinary subprocess or virtual environment would not be a security sandbox.

Follow-up: What would execution need? A separate isolation boundary, no credentials, resource/network/filesystem restrictions, trusted tests, and structured result handling. None is implemented here. **Proof:** report instruction all code is untested; UI caption in `app.py`.

Q18. What do the tests actually demonstrate?

Answer: The recorded baseline is 68 passing mocked tests for guards, contracts, evidence, failures, source/search boundaries, UI, and PDF behavior. Test-only doubles remain even though the runtime demo model was removed.

Follow-up: Do they establish live model quality? No. Synthetic SDK transport tests exercise serialization/status handling without sockets; they do not validate real account access, live latency, current billing, factuality, or scoring fairness. **Proof:** `tests/test_model_errors.py`; `test_harness_accounts_known_failure_usage_once_and_never_retries`; headless AppTest checks.

8 Sources, security, testing, and PDF

This material describes the implementation currently present in the repository. The suggested alternatives are design reasoning for an interview, not evidence that every alternative was benchmarked or previously prototyped. The 68-test result is the recorded deterministic baseline from the previous verification; it is not proof of live provider access or grading quality.

Explain the security boundary accurately

A useful interview explanation is: "The application lets a user submit study material and lets a model produce a structured interview and revision notes. I control the network tools, input sizes, accepted response shape, and reporting rules in Python. The model does not receive a general browser, shell, filesystem tool, or authority to execute code."

This is a local hobby application, not a hardened multi-user service. `.streamlit/config.toml` explicitly binds the server to `127.0.0.1`, enables headless mode, and disables Streamlit usage-stat collection. Loopback binding limits access to the local machine; it does not authenticate other local processes or make outbound provider traffic private. There is no authentication, tenant isolation, database, generated-code sandbox, or execution pipeline. The model's Python examples are text in a report. Saying "I sandboxed generated code" would be inaccurate. If code execution were added, ordinary subprocesses or virtual environments would not meet that requirement; it would need a separate isolation design, fixed external tests, resource limits, and secret-free execution.

The important trust boundaries are: user input enters the app; public page text and Brave snippets enter the model context; the model response enters local validation; accepted notes enter the UI and PDF. Each boundary needs checks appropriate to what crosses it. A valid JSON response is not a proof of factual correctness, and an HTTPS response is not a proof that its content is trustworthy.

The public-page loader: where checks happen

`tutor/sources.py:load_source` accepts a full URL, topic, or pasted text. Pasted material substitutes for fetching a page. A supplied URL still receives syntax validation, but when pasted text is present the loader does not request that URL. Topic-only input also avoids a page request. The resulting frozen `Source` dataclass holds title, retained text, source URL, and a truncation flag. It does not store a DOM or browser session.

`_validated_url` rejects unsupported schemes, missing hosts, embedded usernames/passwords, nonstandard ports, control characters, spaces, backslashes, and excessively long URLs. Only HTTP on port 80 and HTTPS on port 443 are allowed. It normalizes an internationalized hostname using IDNA, percent-encodes path/query characters, and removes fragments because a fragment is not sent to the server. This is deliberately narrower than "any URL Python can parse." `file://` is not a study-source transport.

`_public_address` resolves the hostname and checks every returned IP address. A mixture of public and private answers is rejected. IPv4-mapped IPv6 addresses are checked using their underlying IPv4 address. Loopback, private, link-local, multicast, reserved, and other nonglobal addresses are blocked by the `ipaddress` checks. This prevents straightforward attempts to make the server fetch its own local services or cloud metadata addresses.

Checking DNS and then connecting using the hostname would create a check/use gap: a second lookup could return a different address. `_PinnedConnection` instead opens its socket to the already checked IP. For HTTPS it wraps the socket with Python's default TLS context and passes the original hostname as `server_hostname`. The IP fixes the destination; the hostname keeps certificate verification and TLS server-name selection aligned with the requested site. One validated address is selected; the current code does not implement sophisticated fallback across all public addresses.

`_fetch_page` follows at most three redirects. Every redirect target goes through URL validation, and `_request_once` performs public-IP validation for each actual request. A public page redirecting to `127.0.0.1` is therefore rejected before a second connection. Redirect handling is explicit so that a convenience HTTP client's default redirect behaviour cannot silently bypass the checks.

`_request_once` accepts HTML, XHTML, or plain text, requests identity encoding, rejects unsupported compressed responses, and limits a downloaded body to 1,000,000 bytes. It checks Content-Length when present but also counts streamed bytes, because headers may be absent or false. Connections close in a `finally` block. The request timeout is eight seconds and body reads use a shrinking monotonic deadline. Do not call this a guaranteed eight-second end-to-end deadline: DNS lookup occurs before the deadline is created, and receiving response headers is not checked against a strict total deadline. Each redirect is also a separate request.

The HTML parser drops scripts, styles, navigation, forms, and other common noise. It prefers text in `main` or `article`, falling back to other readable text. It extracts a title, normalizes whitespace, and rejects pages with too little meaningful text. Retained material is capped using the caller's character limit; the app's configuration retains a short excerpt. These controls reduce noise and bound prompt size. They do not understand every site's layout. A JavaScript-rendered page, paywall, or login requirement may require pasted material instead. The code never executes page JavaScript.

Alternatives include a headless browser for dynamic pages, a readability library for richer article extraction, and a normal HTTP client with a custom validated transport. Those approaches may improve coverage, but add dependencies or more execution/network behaviour. The present standard-library loader is inspectable and sufficient for a one-page hobby workflow. Its custom networking and parser also carry maintenance responsibility; small does not automatically mean more secure.

Brave is a bounded context tool

`tutor/search.py:BraveSearch.search` calls one fixed HTTPS endpoint, `api.search.brave.com`, rather than fetching a model-selected URL. Queries are normalized and capped at 300 characters and 75 words. A call returns at most three accepted results. Responses are limited to 1,000,000 bytes; snippets are capped at 800 characters each, titles at 160 characters, and combined context at 4,000 characters. The harness controls whether a search is permitted and how many searches the run may spend.

Results contain title, citation URL, and snippet. `_source_url` rejects obvious unsafe citation forms such as non-HTTP URLs, credentials, nonstandard ports, local hostnames, and private literal IP addresses. It deliberately does not resolve arbitrary citation hostnames or open result pages. Therefore a citation should not be described as a fully validated public-fetch destination. If a future feature opens one, it must go through the page loader's stronger retrieval checks.

`_text` strips HTML tags, unescapes entities, normalizes whitespace, and replaces an exact occurrence of the active Brave token. Non-200 provider error bodies are not read, because they might echo sensitive information. Safe `SearchError` messages replace raw transport details. These measures reduce accidental leakage; exact-value replacement is not a general secret scanner or a defence against arbitrary encoded secrets.

The search context is recorded with provenance and passed back into planning. Linked pages are not independently verified. A snippet might be stale, incomplete, misleading, or contain prompt-injection instructions in ordinary prose. Dropping scripts does not remove malicious natural-language instructions. The important boundary is that the harness, rather than retrieved text, owns available tools, budgets, scoring validation, and state transitions. A bounded search improves context without proving it is true.

Alternatives include an autonomous browsing agent, a curated local knowledge base, or retrieval over embeddings. Autonomous browsing increases the action surface and cost. A curated collection improves

control but needs maintenance. Vector retrieval is useful when there is an actual document corpus; the current app has a small source excerpt and optional snippets, so no vector database is necessary.

Runtime credential loading

`tutor/secrets.py:load_key` selects one provider at runtime. With the normal default path, supported environment variables take precedence. Otherwise it reads the local `api_key.txt` format and selects exactly one matching named credential. It supports named lines and JSON aliases, rejects missing or ambiguous matches, caps file size, and validates the selected value. Explicit test paths skip environment lookup, which keeps synthetic fixture behaviour predictable.

The parser may read the entire local key file to find a named entry; saying "the Brave client never reads a file containing an OpenAI key" would be misleading. The useful guarantee is that the caller receives only the selected provider credential. Brave loads its token only when `search` is actually called. The UI constructs external clients after valid input, rather than exposing keys through a form.

`.gitignore` excludes `api_key.txt`, environment files, the virtual environment, Streamlit secrets, and caches. Ignoring a file does not remove an already tracked key from repository history. For this guide no credential value was inspected. A local key file remains plaintext; environment variables also remain process-accessible and can leak through careless tooling. A deployed service should use an appropriate secret store and restricted identity. Never place real keys in sample configuration, screenshots, prompts, exception text, generated code, or PDFs.

The low-complexity choice is environment variables plus an ignored local file for a single-user project. A cloud secret manager provides rotation, access policies, and auditability, but needs infrastructure and identity management absent here. The correct interview claim is "I limited credential exposure in the application," not "I built an encrypted secret vault."

PDF export is presentation, not verification

`tutor/pdf_export.py:build_pdf` receives the accepted notes object, source material, transcript, configuration labels, status, and research provenance. It creates a ReportLab document in a `BytesIO` buffer and returns PDF bytes. The renderer does not ask the model for another explanation. The app downloads those bytes from session state; no report database or persistent report queue is involved.

This matters because the scorecard and PDF can use the same accepted assessment. Separate model calls for UI feedback and PDF feedback could disagree. The PDF includes source material, assumptions, key concepts, scoring evidence, approaches when appropriate, worked examples, complexity claims, mistakes, follow-up sketches, and the transcript. Theory topics can legitimately have no Python approaches. The renderer does not invent code just to fill a template.

Text passed to ReportLab `Paragraph` is escaped before line breaks are inserted. A model-produced string such as `<b>reference</b>` is displayed as content rather than becoming uncontrolled formatting. Code uses plain-text `Preformatted`, expands tabs, and wraps long display lines at 88 characters. The original code string is not executed or rewritten as a solution; wrapped display text should not be copied as guaranteed executable source. Font registration prefers local Arial/Consolas or DejaVu fonts, with built-in Helvetica/Courier fallbacks.

Layout uses A4 pages, fixed margins, readable line spacing, heading keep-with-next rules, repeated table headers, and a page-number footer. These choices reduce orphan headings and difficult tables. They cannot guarantee every adversarially long unbreakable string or unusual Unicode glyph will render perfectly. Content extraction tests and rendered visual checks answer different questions; both are useful when layout changes.

The report always labels code untested and scores as learning feedback rather than hiring readiness. Early completion is labelled partial. Missing final notes produce an incomplete report preserving the available source and transcript. Failure therefore remains visible instead of being converted into confident-looking marks. ReportLab is a practical Python-native choice; HTML-to-PDF gives richer CSS but adds a rendering engine, and LaTeX offers strong typesetting but adds build tooling and escaping obligations for user-controlled content.

Testing: what the evidence actually supports

The six test modules separate failure surfaces. `test_sources.py` checks URL rejection, mixed private/public DNS, IP pinning, TLS hostname verification, unsafe redirects, extraction, truncation, body limits, and safe error messages. `test_search.py` checks the fixed endpoint, lazy credentials, query/result limits, unsafe citation filtering, malformed responses, no retries, and credential redaction. `test_secrets.py` uses temporary synthetic files and patched environments to test provider separation, aliases, duplicate rejection, size limits, and environment precedence.

`test_research.py` exercises the orchestration of optional search: stable topics skip it; fresh context triggers one bounded replan; a second research request cannot repeat the search; failed searches are counted and not retried; insufficient budgets stop before spending; snippet text cannot be used as learner evidence. `test_model_errors.py` exercises failure categories, incomplete/refused/invalid responses, strict schemas, usage accounting, safe error presentation, and real SDK serialization through an in-memory `httpx2.MockTransport`. That test uses a synthetic key and blocks socket creation. It verifies adapter behaviour without paying or connecting to OpenAI.

`test_tutor.py` covers input limits, plan/score validation, exact evidence in one learner answer, terminal behaviour, accounting, PDF contents, and headless Streamlit user flows. Its `NotesModel` is a test fixture; it is not a selectable demo application mode. Removing the old demo feature does not mean removing deterministic test doubles.

The recorded baseline is 68 deterministic tests passing. This supports repeatable behaviour for the covered examples. It does not verify account quota, real model availability, live Brave access, actual provider billing, grading fairness, semantic correctness of generated Python, full correctness of complexity claims, or all PDF layouts. Additional live smoke tests should be small and explicitly budgeted. Calibrating grading needs independently reviewed answers and scores, not just a passing schema test.

9 Security and testing interview questions

Q19. Why reject a URL that points to localhost?

Answer: The request runs from the application's machine, so arbitrary destinations could expose local services or metadata.

Harder probe: A public hostname is not enough: validate every resolved address and each redirect.

Code to open: `sources.py:_public_address`, `test_sources.py:test_private_dns_addresses_and_mixed_answers_are_rejected`.

Q20. Why pin an IP after DNS validation?

Answer: It prevents a second DNS lookup from changing the checked destination.

Harder probe: Pinning is not content verification and the current client selects only one accepted address.

Code to open: `sources.py:_PinnedConnection.connect`, pinning test.

Q21. Does connecting to an IP disable certificate checks?

Answer: No: the connection uses the original hostname for default TLS verification.

Harder probe: Never "solve" a certificate failure with disabled verification; reject or use pasted material.

Code to open: `_PinnedConnection.connect`, TLS mock assertion in `test_sources.py`.

Q22. Can a redirect bypass the policy?

Answer: Each redirected URL and destination is rechecked.

Harder probe: The three-redirect bound limits chains, but a request still has its own timeout and network overhead.

Code to open: `_fetch_page`, redirect-to-private test.

Q23. Is eight seconds a strict whole-fetch timeout?

Answer: No; there are socket timeouts and a shrinking body-read deadline.

Harder probe: DNS and header waits need additional deadline-aware design for a strict total cap.

Code to open: `_public_address`, `_request_once`.

Q24. Why check streamed bytes if Content-Length is available?

Answer: It may be missing or inaccurate.

Harder probe: Rejecting compressed responses also avoids an unbounded decompression path, but reduces compatibility.

Code to open: `_request_once`, download-limit tests.

Q25. Why not use a browser for all pages?

Answer: Static HTML/text covers the intended simple workflow with fewer moving parts.

Harder probe: JavaScript-only sites require a pasted fallback; a browser would improve coverage but expand dependencies and execution behaviour.

Code to open: `_PageText`, `_fetch_page`.

Q26. Are Brave snippets trusted knowledge?

Answer: No: they are bounded context with provenance and explicit verification limitations.

Harder probe: HTTPS protects transport; it does not validate a snippet's factual claims or remove prompt injection.

Code to open: `search.py:SearchContext`, `pdf_export.py:build_pdf`.

Q27. Do you open all search results?

Answer: No, result URLs are citation metadata.

Harder probe: Citation validation skips DNS, so future retrieval must use the stronger page-fetch policy.

Code to open: `search.py:_source_url`, `BraveSearch.search`.

Q28. Where do credentials enter the program?

Answer: Provider-specific runtime loading, environment first in normal use, with a named local file fallback.

Harder probe: The file parser reads the file to select a value; it is not per-line OS isolation or an encrypted vault.

Code to open: `secrets.py:load_key`, `test_secrets.py`.

Q29. Does `.gitignore` make a leaked key safe?

Answer: No: it prevents new accidental additions, not existing history or screenshots.

Harder probe: A real exposure requires revocation/rotation and history remediation when relevant.

Code to open: `.gitignore`; this is an operational response, not implemented rotation automation.

Q30. Why use fake models in tests after removing demo mode?

Answer: Test doubles provide deterministic covered responses without cost or secret dependencies.

Harder probe: They cannot establish live model quality; distinguish contract tests from quality evaluation.

Code to open: `test_tutor.py:NotesModel`, `test_model_errors.py`.

Q31. How do you know SDK tests do not call the internet?

Answer: The SDK is backed by an in-memory mock transport, synthetic credentials, and a blocked socket constructor.

Harder probe: A patched high-level method alone would not exercise serialization; MockTransport does.

Code to open: `test_model_errors.py:test_sdk_mock_http_response_retains_truncated_output_usage`.

Q32. Can the PDF show a different score from the page?

Answer: Both consume the accepted notes assessment rather than independent model generations.

Harder probe: This prevents renderer divergence, not incorrect model grading.

Code to open: `pdf_export.py:build_pdf`, PDF consistency checks in `test_tutor.py`.

Q33. What happens if final generation fails?

Answer: Preserve source and transcript, show no fabricated marks, and export an explicitly partial report.

Harder probe: Terminal finish behaviour avoids accidental retries on rerenders.

Code to open: `test_tutor.py:test_malformed_final_report_exports_answers_without_retry`, `build_pdf`.

Q34. Are the report's Python solutions verified?

Answer: No; they are unexecuted educational text and visibly labelled untested.

Harder probe: To verify them, add a real sandbox and independent immutable test oracle; do not run model code directly on the host.

Code to open: `pdf_export.py` untested labels, absence of an execution component.

10 Technology and system design interview questions

Q35. Why Python, and does the GIL make it unsuitable?

Answer: Python keeps the UI, provider adapter, validation, tests, and rendering in one familiar language. This workload mostly waits for external I/O. CPU throughput is not a demonstrated bottleneck, so the GIL is not a reason to change languages now.

Harder probe: Threads can overlap some I/O, but this workflow has dependent calls and mutable session state. Async syntax does not automatically improve it. If many users need concurrent sessions, measure latency and throughput, then separate request handling from bounded background work. This project has no load-test evidence.

Code to open: `app.py`, `TutorSession.start`, `OpenAIModel.generate`.

Q36. Why Streamlit rather than React and FastAPI?

Answer: The requirement is a small local Python app. Streamlit provides input, chat, session state, and downloads without maintaining a separate frontend or HTTP application API.

Harder probe: Streamlit reruns require explicit event guards and cached accepted results. A separate frontend would improve complex interaction and client flexibility, but introduces deployment, API, and authentication work. I would change when those requirements justify it, not because a framework sounds more professional.

Code to open: the form submission, `st.session_state`, and terminal PDF branch in `app.py`.

Q37. Where is the machine learning in this project?

Answer: It consumes a pretrained hosted language model at inference time. The application work is orchestration and evaluation around that model. It does not train, fine-tune, or implement a classifier.

Harder probe: Adding supervised-learning topics to the interview does not mean the app itself implements supervised training. A vector database would support retrieval over a corpus, not train the model. Explain inference, retrieval, and training as separate operations.

Code to open: `OpenAIModel.generate`, `PLAN_INSTRUCTIONS`, `REPORT_INSTRUCTIONS`.

Q38. How would you choose a different model responsibly?

Answer: Use a small independently reviewed evaluation set covering coding, theory, current topics, ambiguous sources, and partial answers. Compare valid-plan rate, useful-question coverage, grading agreement, failure rate, latency, and cost. Preserve the same rubric and constraints for a fair comparison.

Harder probe: A cheaper token price can still mean a higher cost per useful report if more responses fail. Quality and price must be measured together. Current model prices are verified references; comparative quality and live account access remain unverified. Changing a provider requires more than changing a model string.

Code to open: `Config`, `_response_format`, and the model contract tests.

Q39. How would you measure whether marks are fair?

Answer: Have knowledgeable reviewers label answers using the same rubric. Include correct, partially correct, incorrect, terse, verbose, and misleading answers. Measure category agreement, mean absolute score error, unassessed cases, and reviewer disagreement. Inspect whether evidence actually supports the rationale.

Harder probe: Expert labels are imperfect too; adjudicate disagreements and record uncertainty. Stratify by topic and level and avoid using verbosity as a proxy for understanding. Exact quotes and integer bounds are provenance checks, not this quality study. No such calibration is implemented yet.

Code to open: `SCORING_RUBRIC`, `InterviewAssessment.validate_evidence`, and scoring fixtures.

Q40. What belongs in unit tests versus model evaluations?

Answer: Unit tests verify deterministic rules: URL rejection, state transitions, budget guards, evidence matching, and output rendering. Integration tests verify component contracts and SDK transport behavior. Model evaluations assess relevance, correctness, and grading quality on reviewed examples.

Harder probe: A mock returning perfect notes proves the app handles that fixture; it does not prove the live model usually produces perfect notes. A live smoke test proves limited connectivity and one example, not broad quality. Keep these evidence levels distinct.

Code to open: `tests/test_tutor.py`, `test_model_errors.py`, `test_sources.py`.

Q41. Why no SQLite, and what would the schema look like later?

Answer: In-memory state is enough for one local session. A future saved-history feature could use SQLite with `runs`, `questions`, `answers`, and `model_attempts`. A run stores status and configuration; attempts store purpose, reservation, known usage, and safe outcome. Answers link to their question and run.

Harder probe: Persist transitions before dispatch, protect transcript privacy, define retention/deletion, and record unknown outcomes after crashes. A transaction cannot atomically include an external provider call. These tables are a proposal, not existing project storage.

Code to open: `TutorSession.__init__` shows the current fields that would need persistence.

Q42. What does idempotency mean here?

Answer: Repeating `finish()` on the same terminal session does not generate another report. The UI also reuses stored PDF bytes. This protects ordinary rerenders within the running process.

Harder probe: A crash after provider acceptance but before local persistence remains ambiguous. Durable idempotency needs request identities, stored attempt states, recovery policy, and provider-supported behavior where available. Never promise exactly-once external execution just because one method has a guard.

Code to open: `TutorSession.finish`, the PDF cache in `app.py`.

Q43. Does the code follow SOLID or use design patterns?

Answer: It separates UI, orchestration, provider calls, schemas, network retrieval, and rendering. Constructor injection is the most useful pattern: tests provide an object with the same `generate` contract. It is a practical separation of responsibilities, not proof that every design principle is perfectly implemented.

Harder probe: The injected model currently uses a convention rather than an explicit `Protocol`. A protocol could document the interface for type checkers; it would not validate runtime responses. Avoid adding abstractions solely to name more patterns.

Code to open: `TutorSession.__init__`, `ModelReply`, `NotesModel` in tests.

Q44. Why use both dataclasses and Pydantic models?

Answer: Dataclasses suit small internal values and defaults. Pydantic handles external model responses needing runtime validation and JSON Schema. They serve different boundaries.

Harder probe: A type annotation on `ModelReply` does not enforce its content at runtime, so `_request` checks the expected class. Frozen dataclasses prevent attribute reassignment but are not deeply immutable. Configuration still needs stronger validation if exposed to arbitrary users.

Code to open: `Config`, `Source`, `ModelReply`, `_request`.

Q45. How would you add truly adaptive questions?

Answer: Replace the fixed next-question selection with a bounded response contract referencing the last answer and the remaining category coverage. Record each accepted question, reject duplicates, and reserve the final review before each turn. Keep the internal cap and avoid visible planned totals.

Harder probe: Store the evidence that motivated a probe, disallow multiple hidden questions in one turn, and define what happens if a turn fails. This adds latency, spending, and failure opportunities. It is a design change, not something the current `send()` already does.

Code to open: `current_question`, `send`, and `InterviewPlan` show the current fixed-plan design.

Q46. How would you add a code runner?

Answer: Treat it as a separate security-sensitive component. A constrained worker should receive code and inputs without host credentials, have no network, enforce CPU/memory/time/output limits, and restrict its filesystem. Expected outputs and evaluation rules stay outside model edits.

Harder probe: A subprocess or virtual environment is not a security sandbox. Containers also need a reviewed configuration and host boundary. Distinguish wrong answers from timeouts and resource failures, cap attempts, and generate notes from the final tested code version. None of this execution pipeline exists today.

Code to open: current `Approach.code` and PDF untested labels; there is intentionally no executor to demonstrate.

Q47. What changes before hosting this for many users?

Answer: Add authentication, user/run isolation, per-user and global quotas, private durable storage, careful retention, and stronger operational monitoring. Review the network egress policy and separate long-running work from request handling where needed.

Harder probe: Shared provider limits require global rate control; browser session state is not a tenant security system. Design cancellation and crash recovery before claiming reliable background execution. Current loopback binding supports local use and is not a hosted security design.

Code to open: `.streamlit/config.toml`, in-memory `TutorSession`, and configured per-session limits.

Q48. What would you cache, and what would you avoid caching?

Answer: The current app caches accepted notes indirectly in the session and caches rendered PDF bytes. A future source cache could include the URL, retrieval time, policy version, and bounded excerpt. A plan cache would also need model, prompt version, level, and source hash.

Harder probe: Freshness-sensitive questions may need short cache lifetimes. Personalized answers and scores require privacy and user isolation; a global cache could leak data. A source hash proves identity of bytes, not truth. There is no cross-session cache implemented.

Code to open: the terminal export branch in `app.py`.

Q49. What happens when an AI helped write the code?

Answer: Be truthful about assistance. Explain the code you can trace, the requirements you chose, the changes you reviewed, and the checks you ran. Demonstrate that you can modify a validator or follow a failure path without relying on a memorized description.

Harder probe: Do not invent personal implementation history or benchmarks. If a function is unfamiliar, open it and reason through it. An honest limitation with a concrete next check is stronger than claiming ownership of details you cannot explain.

Practice: implement one exercise in section 12 and explain every failing case.

Q50. What is the biggest weakness, and what would you improve first?

Answer: The primary product claim is helpful interview feedback, yet live relevance and grading quality are not calibrated. I would evaluate a small reviewed dataset first, fix contract failures revealed by it, and then add bounded adaptivity if it improves learning.

Harder probe: Infrastructure polish does not establish educational usefulness. Prioritize improvements by evidence and user need: quality, reliability, useful interaction, then persistence or scale when requested. Secure code execution is a separate feature with substantial boundary requirements.

Code to open: rubric, evidence validation, mocks, and the absence of live evaluation data.

11 Actual code excerpts and how to explain them

The snippets below are extracted from the current source without importing the application or loading credentials. They are excerpts, not standalone programs. The Markdown retains the original source lines; the PDF may wrap lines for display. Explain the purpose and invariants rather than reciting syntax.

Excerpt 1: Converting usage to an estimate

Location: `tutor/config.py::Config.cost`.

```
def cost(self, input_tokens: int, output_tokens: int) -> float:
        return (
            input_tokens * self.input_price_per_million
            + output_tokens * self.output_price_per_million
        ) / 1_000_000
```

Input and output tokens have different configured unit prices. Multiplying each count by its price and dividing by one million gives an estimate in dollars. The arithmetic has no side effects. Prices must match the selected model. Floating-point arithmetic is acceptable for this small display estimate; exact financial ledgers would benefit from decimal arithmetic. This function does not query billing or enforce quota.

Excerpt 2: Computing the total locally

Location: `tutor/schemas.py::InterviewAssessment.total`.

```
def total(self) -> int | None:
        marks = [getattr(self, field).marks for field, _, _ in SCORING_RUBRIC]
        return None if any(mark is None for mark in marks) else sum(marks)
```

Read the rubric fields in a fixed order, preserving null marks. If any criterion is unassessed, return None; otherwise sum the marks. The property exposes a derived result rather than accepting a model-supplied total. Zero is a valid assessed score and must not be mistaken for missing data. The decorator @property is outside this extracted function body.

Excerpt 3: Checking evidence provenance

Location: `tutor/schemas.py::InterviewAssessment.validate_evidence`.

```
def validate_evidence(self, answers: list[str]) -> None:
        for field, _, _ in SCORING_RUBRIC:
            item = getattr(self, field)
            if item.marks is not None:
                quote = item.evidence.strip()
                if not quote or not any(quote in answer for answer in answers):
                    raise ValueError("Score evidence does not match the learner's own
answers.")
```

Only assessed criteria require quotes. Strip the quote, require it to be nonempty, and check membership in one raw answer using any(...). Do not join answers first: that can create a quotation no single answer contains. This accepts an exact substring; it does not assess whether the quote justifies the mark.

Excerpt 4: Accounting after an attempted call

Location: `tutor/harness.py::TutorSession._record_usage`.

```
def _record_usage(self, reply, reservation: float):
        if reply.input_tokens is not None and reply.output_tokens is not None:
            self.input_tokens += reply.input_tokens
            self.output_tokens += reply.output_tokens
            actual = self.config.cost(reply.input_tokens, reply.output_tokens)
            self.estimated_cost += actual - reservation
        else:
            self.usage_unknown = True
```

The reservation was already charged before dispatch. Known input and output counts replace it using configured-price arithmetic; unknown usage retains it. Tokens are counted once. This method trusts the adapter's usage fields. Replacement adapters must preserve nonnegative integer counts; stronger validation at this boundary would be a useful improvement.

Excerpt 5: Selecting a question from accepted answers

Location: `tutor/harness.py::TutorSession.current_question`.

```
def current_question(self):
        if self.plan is None or len(self.answers) >= len(self.plan.questions):
            return None
        return self.plan.questions[len(self.answers)]
```

The accepted-answer count is the only index. Missing plan or completed plan returns None, avoiding an out-of-range access. This removes an independently mutable question counter. It also makes the design visibly fixed-plan: this function does not inspect the last answer to generate a new question. The @property decorator appears above the function in source.

Excerpt 6: Preserving answers and stopping safely

Location: `tutor/harness.py::TutorSession.send`.

```
def send(self, text: str):
        if self.status != "Interviewing" or self.current_question is None:
            raise ValueError("This interview has finished. Start a new one to
continue.")
        text = text.strip()
        if not text or len(text) > self.config.max_message_chars:
            raise ValueError(f"Use 1-{self.config.max_message_chars} characters per
answer.")
        self.answers.append(text)
        self.messages.append({"role": "user", "content": text})
        # Never silently truncate answers. Preserve the latest answer in a partial
export.
        if self._input_bound(True) > self.config.max_input_bytes:
            self.status = "Partial - unverified"
            self.notice = "Context allowance reached. Your answers are preserved; AI
grading is unavailable."
        elif self.current_question is None:
            self.finish()
        else:
            self._show_question()
```

Check state before accepting input. Reject empty or oversized text, append the raw accepted answer, then evaluate the future report input bound. If total context is too large, preserve the latest answer and stop with

partial status. Completion calls finish once; otherwise show the next planned question. Reordering appends after the bound check would lose the answer that triggered the limit.

Excerpt 7: Connecting to a checked destination

Location: `tutor/sources.py::_PinnedConnection.connect`.

```
def connect(self):
        self.sock = socket.create_connection((self.address, self.port), self.timeout)
        if self.secure:
            try:
                self.sock = ssl.create_default_context().wrap_socket(self.sock,
server_hostname=self.host)
            except Exception:
                self.sock.close()
                raise
```

Connect to the validated IP and port, so the transport does not repeat hostname resolution. HTTPS wraps the socket with the default TLS context and original hostname. On TLS failure close the socket and propagate the failure; do not disable verification. The surrounding loader validates destinations and sanitizes resulting errors.

Excerpt 8: Making the submitted schema strict

Location: `tutor/openai_model.py::_response_format`.

```
def _response_format(schema):
    """Apply strict API object rules to the same Pydantic response contract."""
    value = schema.model_json_schema()

    def make_strict(node):
        if isinstance(node, dict):
            node.pop("default", None)
            if node.get("type") == "object":
                node["additionalProperties"] = False
                node["required"] = list(node.get("properties", {}))
            for child in node.values():
                make_strict(child)
        elif isinstance(node, list):
            for child in node:
                make_strict(child)

    make_strict(value)
    return {"type": "json_schema", "name": schema.__name__, "schema": value, "strict":
True}
```

Start from the Pydantic schema. Recursively remove defaults; require all properties on each object and forbid extra properties. Traverse nested dictionaries and lists, including definitions. Nullable fields can still contain null, but must be present. This controls response shape; local validators and independent quality checks are still required.

12 Coding interview practice

These are proposed interview exercises, not implemented project features or executed solutions.

Exercise A - a public-address policy. Write a pure function that accepts a list of IP strings and either returns an accepted destination or rejects the entire list. Reject mixed private/public answers and handle IPv4-mapped IPv6. Test an empty list, loopback, link-local, multicast, reserved addresses, a malformed string, and multiple public addresses. Explain why testing the hostname alone is insufficient and why a caller must connect to the accepted IP rather than resolve again.

Exercise B - bounded streaming. Implement a helper over a fake response reader that accumulates at most a configurable number of bytes and observes a monotonic deadline. Test an exact-size response, one extra byte, absent Content-Length, a false short Content-Length, many small chunks, and a stalled reader. State that a pure helper cannot impose a deadline on an uninterruptible DNS lookup or a blocking operation that ignores its timeout.

Exercise C - strict evidence provenance. Implement a function checking whether a required evidence quote occurs in one raw learner answer. Test a valid substring, only-source text, only-question text, a quote formed by joining two answers, no answers, and blank evidence. Then explain the limitation: provenance checks do not prove the quoted statement supports the awarded mark or that copied material reflects understanding.

Exercise D - safe partial PDF export. Given a small notes fixture and transcript, produce bytes without network access and assert the output begins with `%PDF`. Extract text to confirm score consistency, partial/unverified labels, code preservation, source provenance, and literal angle brackets. Render a long-code/long-table fixture for visual inspection. Explain that text extraction alone cannot detect clipped content, missing glyphs, or awkward page breaks.

Exercise E: derived marks with missing evidence

Task: Write a pure function accepting four marks and their maxima. Return an integer total only if all marks are assessed. Reject booleans, numeric strings, negative values, and values above the category maximum. Preserve zero. This is a practice function; the project already enforces related rules through its Pydantic contract.

Solution sketch: Validate length, check each item with `type(value) is int` when not None, then validate its inclusive range. After validation return None if any value is None; otherwise sum. Check `[3, 3, 2, 2]`, `[0, 0, 0, 0]`, `[3, None, 1, 1]`, `[True, 1, 1, 1]`, and `[4, 3, 2, 2]`. With a fixed four-element rubric this is constant time; with k criteria it is O(k).

Cross-question: Why not replace None with zero? That changes missing evidence into an observed failure and creates a misleading total.

Exercise F: normalized duplicate detection

Task: Reject questions that become identical after trimming, collapsing whitespace, and case normalization. Keep the displayed original text. Do not change meaning through aggressive rewriting.

Solution sketch: Build a normalized key for each question and insert it in a set. A repeated key rejects the plan. Test differing case, tabs, multiple spaces, a truly different question, and a blank entry. Expected time is O(T) for T total text characters with ordinary hash behavior; memory is proportional to retained normalized keys. A hash-set complexity claim should state its average-case assumption.

Cross-question: Can this detect paraphrases? No. Semantic deduplication would need separate reasoning or similarity checks and new evaluation; do not claim the normalized set solves it.

Exercise G: reservation reconciliation

Task: Model one attempted call with a reservation. Reconcile it once with known input/output usage, or retain it if usage is unknown. Keep attempted-call count independent of response success.

Solution sketch: Create an attempt record with an ID, reserved amount, and settled flag. Known usage changes the estimate by `actual - reserved`; unknown usage retains the reservation. A second reconciliation must not charge or refund again. Test failure with usage, failure without usage, known zero usage, and repeated settlement. Use synthetic numbers and no provider calls.

Cross-question: Does the current project persist such attempt IDs? No; it uses synchronous session accounting. The durable ledger is a proposed extension useful for retries and crash recovery.

Exercise H: terminal-state idempotency

Task: Implement a small session object with interviewing, finishing, finished, and partial states. A repeated finish after a terminal state must not invoke the injected provider. An empty transcript must not request grading.

Solution sketch: Guard permitted entry states before dispatch, set finishing, attempt one request, then set a terminal outcome. A fake provider counter makes duplicate calls observable. Test normal completion, repeated completion, no answers, thrown errors, and preservation of answers after a failure.

Cross-question: What if the process dies while finishing? An in-memory object cannot answer what the external service did. Describe persistence and recovery rather than pretending the guard guarantees exactly-once execution.

How to practise the coding round

Choose one exercise, state the contract, write failing cases, then implement the smallest function. Explain the trust boundary it protects and a limitation it does not solve. Use fixtures with synthetic credentials only. These exercises are interview practice, not a claim that their sketched solutions were executed.

13 Debugging and failure scenarios

The generic model failure shown in the app

The earlier screenshot showed a failed model request, no valid review, and unknown token usage. That screenshot alone does not identify the root cause. It is consistent with several client or provider failures; the displayed reservation is not evidence of a successful billable response or a provider invoice. The current adapter exposes safe categories instead of a raw provider error.

For diagnosis, identify the stage and safe error kind first. Check nonsecret configuration and installed SDK compatibility, then reproduce locally with a deterministic transport fixture. Authentication or account-credit problems require the account owner to check their provider dashboard. The local session budget does not create provider credits. Avoid repeatedly clicking Start to debug uncertain paid requests.

Symptom or trigger	Meaning and implemented behavior	Where to investigate
Blank or invalid source	Reject before provider construction	`load_source`, Start form test
Private URL or redirect	Reject the destination; use pasted material	URL/IP policy and source tests
No static readable text	Page may require JavaScript/login; no fabricated extraction	`_PageText`, `_fetch_page`
Authentication / quota	Safe provider category; partial content preserved	`_failure_kind`, runtime key source / account settings
Timeout / connection	One attempted call; no automatic retry; uncertain usage retained	adapter timeout and model error tests
Output limit / invalid JSON	Keep available usage, reject incomplete contract	`OpenAIModel.generate`
Valid JSON, wrong plan	Local category/count/duplicate checks reject	`InterviewPlan`, `TutorSession.start`
Missing Brave key or failed search	Count attempted search, stop partial, offer pasted context	`_research`, `BraveSearch.search`
Repeated research request	Stop after bounded replan	`TutorSession.start`
Context grows too large	Preserve accepted answer; no final grading call	`TutorSession.send`
Budget exhausted	Stop before another dispatch, preserve available material	`_request`, `_research`
Invalid score quote	Discard assessment, retain accepted educational notes	`finish`, `validate_evidence`
Criterion explicitly unassessed (`marks=None`)	Preserve unassessed criterion; no overall total	`InterviewAssessment.total`
Terminal UI rerender	Reuse notes/PDF, do not buy another report	status guard and PDF cache
PDF clipped or missing glyphs	Rendering defect requiring visual fixture review	`build_pdf`, font selection, rendered pages

A disciplined debugging explanation

Separate input, provider, contract, orchestration, and rendering failures. A malformed model response is not an answer scored zero. A page-fetch failure is not a learner mistake. A missing review is not proof of insufficient knowledge. Use fixed safe errors and synthetic fixtures so debugging does not expose prompts, keys, or private answers.

The application has useful in-memory state and notices, but no persistent sanitized request log, trace IDs, elapsed-time ledger, or monitoring platform. Do not claim those exist. A future trace record should distinguish attempts, accepted outputs, known usage, unknown outcomes, and final state while avoiding secrets and unnecessary transcript retention.

14 Improvements and scaling roadmap

First: validate educational usefulness

Create a small reviewed evaluation set and define success before running it: topic relevance, category coverage, score agreement, evidence support, accurate complexity explanations, and honest uncertainty. Separate coding and conceptual topics. Compare models using the same fixtures and record cost per accepted useful report. A paid evaluation requires a deliberate allowance; no live evaluation was run for this document.

Next: reliability and useful interaction

Strengthen configuration validation for prices and bounds. Add a carefully bounded malformed-response retry only if evaluation justifies it; count every attempt and never retry authentication/quota failures blindly. Add adaptive probing as an approved design change with category and budget coverage. Improve static-page extraction based on observed unsupported pages rather than loading a full browser by default.

Then: saved sessions when needed

Use SQLite for local saved runs with schema versions, safe attempt records, and explicit deletion. Store accepted questions, raw answers, rubric version, model/configuration, and notes/code hashes if reproducibility requires them. Define recovery for interrupted calls. Reproducibility has limits: a stored transcript does not guarantee a hosted model will reproduce the same wording on a new request.

Separate feature: verified code execution

Build an isolated executor, fixed independent checks, structured results, and an explicitly bounded repair loop. Link explanation generation to a final tested code version, ideally by immutable artifact ID or hash. Keep a strict distinction between code that was executed, code that passed supplied tests, and code that is mathematically proven correct. The current project does none of this execution work.

Only with a real hosted requirement: multi-user operation

Add authentication, isolated private storage, durable jobs, global quotas, and monitoring. Make cancellation semantics explicit: hiding a UI request is not cancellation of provider work already accepted. Use deadline-aware networking and review the egress boundary. Performance claims require measurements under representative load. No deployment is performed or implied by this roadmap.

15 Mock interview rounds and revision checklist

Round one: explain the project in five minutes

Give the sixty second pitch. Draw the architecture from section 2. Trace the ordinary two-model-call path and the bounded research path. Explain why fixed planning and in-memory storage fit a hobby app. End with evidence and limitations: 68 recorded mocked tests, no live quality calibration, and no code runner.

Round two: explain code in five minutes

Open `TutorSession.send`, `_request`, and `InterviewAssessment.validate_evidence`. Explain the state checks, preservation of raw answers, reservation before dispatch, and exact-quote rule. Show where Pydantic enforces contract semantics and where the SDK adapter preserves usage before parsing. Write a small test for zero versus None or a quote spanning two answers.

Round three: handle hard questions in five minutes

Discuss DNS rebinding, TLS hostname verification, a failed provider request with unknown usage, and rerun idempotency. Explain why none proves grading truth or exactly-once external execution. Then prioritize one improvement with measurable success rather than promising every future feature at once.

A safe demonstration sequence

Use the existing test suite to demonstrate failure handling without real credentials or spending. Open the historical sample PDF only as a clearly mocked rendering example. For a real application demonstration, confirm runtime credentials privately and use a narrow familiar topic; account access is not established by the guide. Inspect the returned marks and evidence rather than treating a successful response as proof of accuracy.

Revision checklist

- I can explain the user problem and the actual current scope in one minute.
- I can name every module and its boundary without claiming microservices.
- I can trace Start, an intermediate answer, early Finish, and terminal rerender.
- I can distinguish fixed questions from future adaptive follow-ups.
- I can explain JSON validity, schema validity, business rules, provenance, and semantic quality.
- I can explain None versus zero and the locally computed total.
- I can explain failed-attempt accounting and why estimated cost is not exact billing.
- I can explain public-IP checking, pinning, TLS hostname verification, and redirect revalidation.
- I can describe secret loading without showing a real value or implying an encrypted vault.
- I can distinguish SDK mocks, unit tests, live smoke checks, quality evaluation, and visual PDF review.
- I can explain why generated code is untested and what a real sandbox would require.
- I can implement one coding exercise and state its limitations.
- I can discuss alternatives as trade-offs without inventing benchmarks or personal history.

16 Glossary and quick reference

Term	Meaning in this project
Harness	Python code controlling inputs, calls, validation, tools, and stopping
Orchestration	Ordering the plan, optional research, answer collection, and review
Adapter	A boundary translating a provider-specific API into the local contract
Dependency injection	Supplying the model/search implementation to the session
Structured output	A response constrained to the submitted JSON Schema
Semantic validation	Local business checks beyond response field structure
Provenance	Where a quoted answer or reference snippet came from
Model evaluation	Measuring answer quality using independently reviewed cases
Test double	A deterministic stand-in used for covered application checks
Reservation	A conservative estimate charged locally before an attempted request
Unknown usage	Missing token metadata, so the estimate remains conservative
Idempotency	Repetition having no extra effect within the stated boundary
Terminal state	A session outcome after which normal model work stops
SSRF	Server-side requests reaching unintended destinations via untrusted URLs
DNS rebinding	A hostname resolving differently between checking and connecting
IP pinning	Connecting to the already checked destination address
TLS hostname check	Verifying the certificate against the originally requested hostname
Prompt injection	Untrusted material attempting to override trusted task instructions
Inference	Requesting output from a pretrained model without training it
Retrieval	Obtaining relevant context; here a page excerpt or search snippets
Sandbox	An isolated execution boundary; not present for generated code here
Rendering check	Inspecting actual PDF pages, beyond extracted text assertions

Quick answers to common traps

"Strict JSON means correct answers": no, it constrains shape. "A quote makes scores fair": no, it checks provenance. "Unknown usage is free": no, keep the reservation. "A virtual environment is a sandbox": no, it manages dependencies. "Finished means verified code": no, it means the workflow ended. "Loopback means authentication": no, it only limits network binding. "Mock tests establish model quality": no, they establish covered deterministic behavior. "store=False means zero retention everywhere": no, provider policy has other retention rules.

17 Source references and maintenance

Repository evidence

The primary source is the current repository: `app.py`, `tutor/config.py`, `harness.py`, `schemas.py`, `openai_model.py`, `sources.py`, `search.py`, `secrets.py`, and `pdf_export.py`; the six test modules; `.streamlit/config.toml`; and dependency files. Function names are more robust than stale line numbers. The architecture presentation is a complementary view, and `docs/requirements.md` contains the project's evolved scope. Historical sample notes are mocked examples, not live provider output.

This guide was prepared on 5 October 2026 from the existing implementation and the recorded 4 October verification. No credentials were inspected, no model/search calls were made, and no new feature implementation is represented by its proposed exercises or roadmap. Update it when state transitions, contracts, rubric, limits, provider, or scope change.

Official references

- OpenAI GPT-4.1 mini model and prices: model capabilities and configured standard text price reference.
- OpenAI Structured Outputs: schema constraints and separate failure handling.
- OpenAI API data controls: retention distinctions; `store=False` is not a blanket zero-retention claim.
- Streamlit execution flow and Session State: reruns, forms, and session behavior.
- Pydantic model documentation: parsing, validation, and model configuration.
- Brave Search plan pricing: the configured per-search estimate is based on $5 per 1,000 requests; account terms may differ.
- ReportLab Platypus guide: document flowables and rendering design.

Pricing and external capabilities were checked against official pages on 4 October 2026. Recheck them before changing the model or spending policy; availability in a particular account remains a separate question.